Planejamento de trajetória para robô terrestre a partir de imagens aéreas usando grafos

Path planning for a ground robot from aerial images using graphs

Gabriel Dias Vilela, Tassandro Cavalcante Leitão, Claudia Marcela Justel e Paulo Fernando Ferreira Rosa

Instituto Militar de Engenharia (IME), Seção de Engenharia de Computação (SE/9) – Praça General Tibúrcio, 80, Praia Vermelha, CEP 22290-270, Rio de Janeiro, RJ, Brasil

dias.gabriel@ime.eb.br, leitao.tassandro@ime.eb.br, justel@ime.eb.br, rpaulo@ime.eb.br

RESUMO: Este trabalho trata do planejamento de trajetória para veículos terrestres não tripulados (UGV) a partir de imagens aéreas obtidas por veículos aéreos não tripulados (UAV). Foi desenvolvido um sistema modular e orientado a objetos, com cinco estágios encadeados: ingestão de dados, pré-processamento, construção de grafos, algoritmos e pós-processamento. As áreas transitáveis são identificadas por segmentação semântica com redes U-Net pré-treinadas no Semantic Drone Dataset, e o mapa resultante é convertido em um grafo cujos pesos penalizam a proximidade de obstáculos. Sobre esse grafo, são comparados os algoritmos BFS, DFS, Dijkstra e A* e o planejador por amostragem RRT. O caminho é enviado pelo ROS 2 a um UGV Clearpath Jackal simulado no Gazebo, e um conversor gera o mapa de ocupação a partir da descrição do mundo simulado, garantindo a correspondência métrica entre planejamento e execução. Os ensaios usaram imagens de drone do Outeiro da Glória, no Rio de Janeiro, e um mundo simulado de galpão com prateleiras e corredores estreitos. O A* e o Dijkstra produziram caminhos de mesmo custo mínimo, com o A* explorando menos vértices; o RRT planejou de 9 a 43 vezes mais rápido, porém com caminhos irregulares. O veículo simulado concluiu os três percursos ensaiados, inclusive sobre imagem aérea real, e seguiu a trajetória planejada com erro lateral médio de 0,8 cm. A validação com veículo físico permanece como etapa seguinte.

PALAVRAS-CHAVE: Veículos terrestres não tripulados. Segmentação semântica. Planejamento de trajetória. Algoritmos em grafos. ROS 2.

ABSTRACT: This work addresses path planning for unmanned ground vehicles (UGV) from aerial images acquired by unmanned aerial vehicles (UAV). A modular, object-oriented system was developed, with five chained stages: data ingestion, preprocessing, graph construction, algorithms, and post-processing. Traversable areas are identified by semantic segmentation with U-Net networks pre-trained on the Semantic Drone Dataset, and the resulting map is converted into a graph whose weights penalize proximity to obstacles. On this graph, the BFS, DFS, Dijkstra, and A* algorithms and the sampling-based RRT planner are compared. The path is sent through ROS 2 to a Clearpath Jackal UGV simulated in Gazebo, and a converter builds the occupancy map from the description of the simulated world, ensuring metric correspondence between planning and execution. The trials used drone images of Outeiro da Glória, in Rio de Janeiro, and a simulated warehouse world with shelves and narrow aisles. A* and Dijkstra produced paths with the same minimum cost, with A* exploring fewer vertices; RRT planned 9 to 43 times faster, but with irregular paths. The simulated vehicle completed the three trials, including one over a real aerial image, and followed the planned trajectory with a mean cross-track error of 0.8 cm. Validation with a physical vehicle remains the next step.

KEYWORDS: Unmanned ground vehicles. Semantic segmentation. Path planning. Graph algorithms. ROS 2.

1 Introdução

Os avanços em visão computacional, eletrônica e inteligência artificial impulsionaram o emprego de robôs autônomos em ambientes de combate e em missões de resgate em eventos extremos, como terremotos, chuvas intensas e acidentes com material radioativo, nos quais eles permitem atuar em áreas perigosas reduzindo a exposição humana. Para a autonomia de um veículo terrestre não tripulado (UGV, do inglês *Unmanned Ground Vehicle*), uma das tarefas essenciais é o planejamento de trajetória: determinar, a partir de uma representação do ambiente, um caminho livre de obstáculos entre a posição atual e o destino.

Uma forma de obter essa representação é empregar um veículo aéreo não tripulado (UAV, *Unmanned Aerial Vehicle*) para fotografar a área de operação. A cooperação entre plataformas aéreas e terrestres é objeto de pesquisa em andamento no laboratório LIARC do Instituto Militar de Engenharia (IME) [1]. Nessa abordagem, a imagem aérea é segmentada para identificar as regiões transitáveis, e o mapa resultante é modelado como um grafo, cujos vértices representam posições transitáveis e cujas arestas ligam posições vizinhas; algoritmos de busca em grafos calculam, então, o caminho.

Um Projeto de Final de Curso anterior [2] demonstrou a viabilidade dessa abordagem com os algoritmos de Dijkstra e A* sobre imagens aéreas já segmentadas, comparando-os com o planejador por amostragem RRT. Aquele trabalho, contudo, dependia de mapas previamente segmentados por limiarização de cor e não contemplava a execução do caminho por um veículo. O presente trabalho dá continuidade a essa linha e leva o planejamento da imagem aérea real até o movimento de um UGV, ainda que em ambiente simulado.

As principais contribuições são: (i) uma arquitetura de *software* modular, em que modelos de segmentação e algoritmos de planejamento são intercambiáveis; (ii) a segmentação semântica de imagens reais de drone dentro do próprio *pipeline*; (iii) a comparação de cinco planejadores sobre os mesmos grafos; (iv) a execução da trajetória em ciclo fechado por um UGV simulado, via ROS 2 e Gazebo; e (v) a geração automática do mapa de ocupação a partir da descrição do mundo simulado. As seções seguintes apresentam os objetivos, a justificativa, a metodologia, os resultados e as conclusões.

2 Objetivos

O objetivo geral é construir um sistema modular capaz de, a partir de uma imagem aérea real, identificar as áreas transitáveis, planejar uma trajetória por diferentes algoritmos e executá-la em um veículo terrestre, validando-a previamente em ambiente simulado. Os objetivos específicos são:

a) projetar e implementar uma arquitetura orientada a objetos com cinco módulos encadeados: ingestão de dados, pré-processamento, construção de grafos, algoritmos e pós-processamento;
b) implementar a segmentação semântica de imagens RGB de drone com modelos pré-treinados [3], gerando mapas de navegabilidade;
c) incorporar o RRT ao catálogo de algoritmos, sob a mesma interface de BFS, DFS, Dijkstra e A*, e comparar os cinco sobre os mesmos mapas;
d) integrar o planejamento ao ROS 2 e executar a trajetória em um UGV simulado (Clearpath Jackal) no Gazebo;
e) caracterizar os percursos simulados e tratar a correspondência métrica entre a imagem de planejamento e o mundo simulado.

3 Justificativa

O trabalho insere-se na pesquisa de cooperação entre UAV e UGV conduzida no IME [1] e pode ser aproveitado no projeto estratégico Sistemas de Veículos Terrestres Remotamente Pilotados (SVTRP), do Centro Tecnológico do

Exército (CTEx). Três lacunas do trabalho anterior [2] motivam o projeto.

A primeira é a segmentação. A limiarização por cor funciona quando o terreno transitável tem cor uniforme e distinta, mas falha em imagens reais, nas quais sombras, variações de iluminação, pavimentos de tonalidades diferentes e telhados de cor semelhante ao piso confundem o limiar. Modelos de segmentação semântica treinados especificamente sobre imagens de drone [3] tratam esses casos e podem ser reutilizados sem retreinamento, o que mantém a coerência com a linha de pesquisa do laboratório.

A segunda é a comparação entre planejadores. Algoritmos determinísticos de busca em grafos garantem o caminho de menor custo, enquanto planejadores por amostragem, como o RRT, são rápidos e dispensam a construção do grafo. Avaliá-los sobre os mesmos mapas e com as mesmas restrições permite discutir objetivamente esse compromisso.

A terceira é a execução. Um caminho ótimo sobre uma imagem não garante que um veículo consiga segui-lo. A simulação no Gazebo [4], com as mesmas interfaces ROS 2 [5] que seriam usadas em um robô físico, permite fazer essa verificação sem risco e sem depender de uma plataforma real, separando os erros de planejamento dos erros de execução antes de qualquer ensaio em campo.

4 Metodologia

4.1 Arquitetura do sistema

O sistema foi implementado em Python e organizado em cinco módulos independentes e encadeados (Fig. 1). Cada módulo define interfaces abstratas e uma fábrica de registro (*Factory Pattern*), de modo que trocar o modelo de segmentação, o algoritmo de planejamento ou o formato de saída exige apenas registrar uma nova classe, sem alterar os demais módulos. O módulo `data_ingestion` estrutura a imagem de entrada; o `preprocessing` aplica a segmentação semântica; o `graph_parsing` converte o mapa de navegabilidade em grafo; o `algorithm` executa o planejador escolhido e devolve um resultado padronizado, com a sequência de vértices, o custo e metadados de execução; e o `post_processing` converte esse resultado em *waypoints*, desenha os caminhos sobre a imagem e, por meio de uma ponte, publica a imagem e a trajetória no ROS 2.

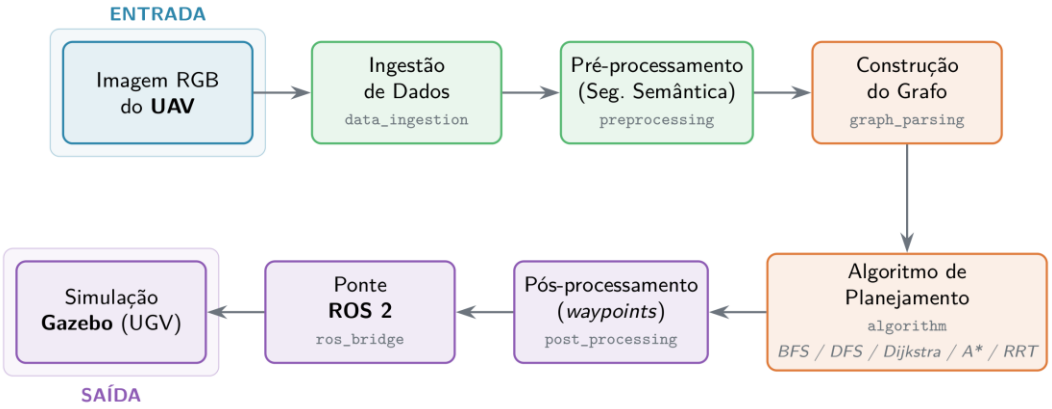


Fig. 1 – Pipeline do sistema, da imagem RGB capturada pelo UAV até a simulação do movimento do UGV no Gazebo.

4.2 Segmentação semântica

A identificação das áreas transitáveis usa os modelos disponibilizados em [3]: redes U-Net [6] com *backbones* ResNet 50/101, EfficientNet-B4/B5 ou MobileNet V2, treinadas sobre o *Semantic Drone Dataset* [7], conjunto de imagens aéreas anotadas pixel a pixel em 23 classes e capturadas de 5 a 30 m de altura. A imagem é convertida para RGB, normalizada com a média e o desvio-padrão da ImageNet e preenchida até o múltiplo de 32 mais próximo, exigido pelo codificador; a classe de cada pixel é a de maior probabilidade. As classes de solo (*paved-area*, *dirt*, *grass* e *gravel*) são agrupadas como transitáveis e as demais como obstáculos, o que gera o mapa de navegabilidade. A escolha do *backbone* é um hiperparâmetro do *pipeline*.

4.3 Construção do grafo e função de custo

Cada pixel transitável do mapa torna-se um vértice, ligado aos seus vizinhos imediatos (conectividade 8). Seja $d(v)$ a distância euclidiana, em pixels, do vértice v ao obstáculo mais próximo, obtida em uma única passagem pela transformada de distância; sejam r a folga mínima exigida, que representa o raio do robô acrescido de uma margem, e L o raio de influência dos obstáculos. A penalidade de cada vértice é

$$p(v) = \infty, \text{ se } d \le r;\ L - d, \text{ se } r < d < L;\ 0, \text{ se } d \ge L \qquad (1)$$

Vértices com $d(v) \le r$ são descartados, pois o veículo não caberia ali; entre r e L, a penalidade cresce à medida que o vértice se aproxima do obstáculo, o que encarece, sem proibir, os caminhos rentes às bordas. O peso da aresta $e = (u, v)$ soma o comprimento geométrico do passo, $\delta(e) \in \{1, \sqrt{2}\}$, à penalidade do vértice de chegada:

$$w(e) = \delta(e) + p(v) \qquad (2)$$

Como a penalidade depende do vértice de chegada, o grafo é direcionado. Os atributos dos vértices, a lista de arestas e os atributos das arestas ficam em estruturas numéricas separadas, e as arestas são construídas com operações vetorizadas sobre a matriz da imagem, cerca de cem vezes mais rápidas do que laços sobre cada pixel.

4.4 Algoritmos de planejamento

Foram implementados, sob a mesma interface, quatro algoritmos de busca em grafos [8] e um planejador por amostragem. A busca em largura (BFS) encontra o caminho com o menor número de arestas, sem considerar os pesos; a busca em profundidade (DFS) é mantida como referência de busca sem critério de custo; o algoritmo de Dijkstra [9] e o A* [10], este com heurística euclidiana, encontram o caminho de menor custo ponderado. O RRT (*Rapidly-exploring Random Tree*) [11] cresce uma árvore a partir da origem, amostrando pontos do espaço livre e avançando um passo fixo de 20 px em sua direção, com viés para o destino. Ele opera em coordenadas contínuas, respeita a mesma folga r dos demais e verifica colisões amostrando cerca de 100 pontos ao longo de cada segmento, como na implementação de referência [12]. Seu custo reportado é o comprimento geométrico do caminho, não comparável ao custo ponderado dos demais.

4.5 Integração com ROS 2 e Gazebo

O pós-processamento publica a imagem anotada e o caminho em tópicos do ROS 2 [5] (Fig. 2). Como os *waypoints* são calculados em pixels, a ponte os converte para o referencial do mundo pela distância amostral no

terreno (GSD, *Ground Sampling Distance*), tamanho em metros do lado de um pixel, e pela posição (x_0, y_0) do pixel de origem:

$$x_m = x_{px} \cdot \mathrm{GSD} + x_0, \quad y_m = -y_{px} \cdot \mathrm{GSD} + y_0 \qquad (3)$$

O sinal negativo decorre de o eixo vertical da imagem crescer para baixo. Como o processo de planejamento publica uma única vez e encerra, adota-se a política de QoS *transient local*, que retém a última mensagem para assinantes que se conectem depois.

No robô, o nó `path_follower_node` segue os *waypoints* a partir da pose (x, y, θ) fornecida pela odometria. A cada ciclo, calcula o erro de proa e_θ entre a orientação do robô e a direção do *waypoint* corrente, normalizado em (−π, π], e comanda

$$\omega = \mathrm{sat}(2e_\theta, \omega_{max}), \quad v = v_c \cdot \max(0, 1 - |e_\theta|/(\pi/2)) \qquad (4)$$

com ω_{max} = 1,0 rad/s e velocidade de cruzeiro v_c = 0,4 m/s; a velocidade linear se anula quando o desalinhamento supera 90°. Um *waypoint* é considerado alcançado a menos de 0,12 m (0,15 m para o destino). A plataforma de demonstração é o UGV Clearpath Jackal simulado no Gazebo [4], cuja odometria é fornecida por um filtro de Kalman estendido que funde a odometria das rodas e a unidade inercial.

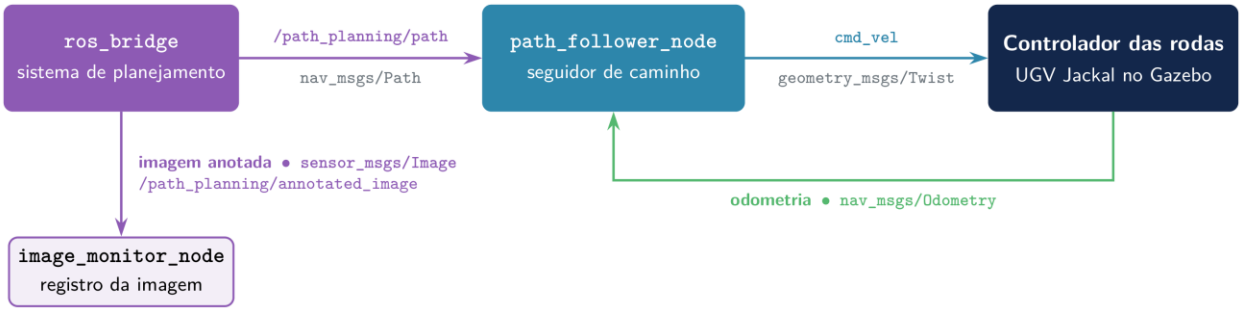


Fig. 2 – Nós ROS 2 e tópicos envolvidos na execução da trajetória. A ponte publica o caminho, que o seguidor converte em comandos de velocidade; a odometria do veículo fecha a malha de controle.

4.6 Mapa de ocupação do mundo simulado

Planejar sobre uma imagem sem relação métrica com o mundo simulado torna o resultado dependente do ajuste manual do GSD e da origem. Por isso, foi desenvolvido um conversor que lê a descrição do mundo do Gazebo no formato SDF, compõe as poses de modelos, elos e elementos de colisão e projeta caixas, cilindros e malhas no plano horizontal, descartando as faces quase horizontais (pisos e tetos). O resultado é um mapa de ocupação no mesmo formato consumido pelo *pipeline*, acompanhado dos valores exatos de resolução e origem. Assim, o caminho é planejado sobre a geometria real do ambiente em que o robô se move, o que separa os erros do planejador dos erros de alinhamento entre referenciais.

4.7 Ambiente experimental

As imagens aéreas são do Outeiro da Glória, no Rio de Janeiro, capturadas por um drone DJI Mavic 3T em vista nadiral; a imagem do percurso simulado tem 4000 × 3000 pixels e foi obtida a cerca de 41 m de altura, acima da faixa do conjunto de treinamento. O mundo simulado é o *warehouse*, galpão de cerca de 30 × 50 m com prateleiras paralelas e corredores, distribuído com o simulador do Jackal. A segmentação foi avaliada com GPU integrada Apple Silicon; o planejamento sobre o mapa de ocupação e a simulação, em processador Intel Core i7-1265U com 16 GB de memória, sem GPU, com ROS 2 Humble e Gazebo sobre Ubuntu 22.04 em WSL2, com o simulador em modo *headless*.

5 Resultados e Discussão

5.1 Segmentação semântica

A Tabela 1 reúne a mIoU e a acurácia divulgadas em [3] para as imagens de teste do *Semantic Drone Dataset* e, medidos neste trabalho sobre uma imagem do Outeiro da Glória redimensionada para 1024 × 768 pixels, a fração da imagem classificada como transitável e o tempo de inferência.

Tabela 1: Comparação dos modelos de segmentação

Modelo	mIoU	Acurácia	Área transit. (%)	Tempo (s)
ResNet 50	0,3933	0,8244	65,5	0,72
ResNet 101	0,4032	0,8720	67,5	0,52
EfficientNet-B4	0,5012	0,9120	61,1	0,71
EfficientNet-B5	0,4819	0,9025	69,4	0,57

O EfficientNet-B4 tem a maior mIoU (0,5012), e todos os tempos ficam abaixo de 1 s, o que faz da mIoU o principal critério de escolha. A dispersão da área transitável, de 61,1% a 69,4%, mostra que o *backbone* altera de forma perceptível o espaço livre entregue ao planejador: um modelo mais permissivo pode incluir regiões de risco, e um mais conservador pode fechar corredores estreitos. Na Fig. 3, o pátio pavimentado e a vegetação rasteira são reconhecidos como transitáveis, enquanto a igreja, as edificações, as árvores e os veículos são excluídos, com bordas que acompanham os limites reais mesmo em regiões sombreadas, principal dificuldade da limiarização de cor.

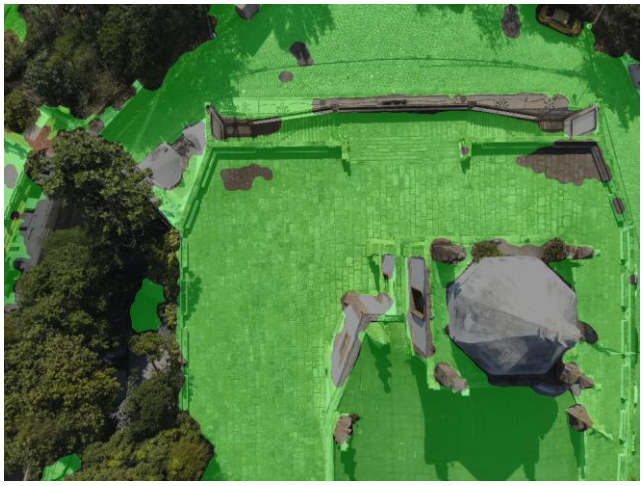


Fig. 3 – Segmentação com o EfficientNet-B4 sobre imagem aérea do Outeiro da Glória; em verde, a região classificada como transitável.

5.2 Algoritmos de planejamento

A Tabela 2 compara os cinco algoritmos sobre o grafo de uma imagem do Outeiro da Glória de 1000 × 750 pixels,

com cerca de 4,4 × 10^5 vértices e 3,3 × 10^6 arestas, e com os mesmos pontos de origem e destino. A distância é a soma dos comprimentos geométricos dos passos; a coluna de exploração traz os vértices explorados pelos algoritmos de busca e as iterações do RRT.

Tabela 2: Algoritmos sobre imagem aérea (1000 × 750 px)

Algoritmo	Vértices	Custo	Explor.	Tempo (s)	Dist. (px)
BFS	351	12.838,8	179.546	0,131	398,0
DFS	65.282	1.314.372,7	248.479	0,159	92.241,7
Dijkstra	879	1.462,2	228.525	0,392	990,7
A*	879	1.462,2	203.472	0,426	990,7
RRT	30	931,6*	289	0,010	931,8

** Para o RRT, comprimento geométrico do caminho.*

O A* e o Dijkstra produziram caminhos ótimos de mesmo custo (1.462,2), com o A* explorando cerca de 11% menos vértices. Ainda assim, o A* não foi mais rápido, porque o cálculo da heurística encarece cada expansão; a vantagem cresce com o tamanho do grafo. A BFS percorre a menor distância (398,0 px), mas, por ignorar a penalização, passa rente aos obstáculos e tem custo ponderado quase nove vezes maior. A DFS gera um caminho cerca de cem vezes mais longo e confirma-se inadequada. O RRT foi o mais rápido (0,010 s, com 289 iterações) e, nesta cena aberta, convergiu quase em linha reta com 30 vértices (Fig. 4).



Fig. 4 – Detalhe dos caminhos sobre imagem aérea do Outeiro da Glória. Em amarelo, a região não transitável; partida em vermelho e chegada em verde.

No mapa de ocupação do *warehouse* (Tabela 3), com prateleiras e corredores estreitos, o grafo tem 502.833 vértices e 3.753.214 arestas (863 × 1242 pixels, 0,05 m por pixel); o tempo é a mediana de três execuções.

Tabela 3: Algoritmos sobre o mapa de ocupação do *warehouse*

Algoritmo	Vértices	Custo	Explor.	Tempo (s)	Dist. (px)
A*	609	28.714,2	391.239	2,173	709,1
Dijkstra	609	28.714,2	397.659	1,694	709,1
BFS	599	44.174,4	414.703	0,674	722,3
DFS	2.614	233.982,4	467.072	0,843	3.693,7
RRT	23	758,5*	230	0,073	757,3

** Para o RRT, comprimento geométrico do caminho.*

Novamente o A* e o Dijkstra empataram em custo, contornando as prateleiras pelo meio dos corredores. A BFS seguiu um trajeto distinto, rente às prateleiras, com custo 54% maior, e a DFS produziu um caminho cinco vezes mais longo. O RRT planejou cerca de 30 vezes mais rápido que o A*, com comprimento próximo (757,3 px contra 709,1 px), mas formado por segmentos curtos com mudanças bruscas de direção, consequência da amostragem aleatória. Para um veículo de tração diferencial, que reduz a velocidade a cada mudança de direção, esse traçado pode levar mais tempo para ser percorrido do que um caminho mais longo e suave; por isso, o A* ponderado foi adotado nos ensaios de simulação.

5.3 Execução no simulador

Foram realizados três ensaios no mundo *warehouse*, todos com o caminho planejado pelo A* e concluídos com o robô alcançando o destino: (i) um percurso de referência, em linha reta de cerca de 20 m (281 *waypoints*) sobre um mapa sintético; (ii) um percurso planejado sobre a imagem aérea real do Outeiro da Glória, segmentada pela U-Net com *backbone* MobileNet V2; e (iii) um percurso planejado sobre o mapa de ocupação do próprio mundo, com folga de 0,6 m. Nos dois primeiros, a imagem não corresponde à geometria do mundo, e o ensaio avalia o seguimento do caminho; no terceiro, o caminho desvia dos obstáculos reais do galpão.

No terceiro ensaio (Fig. 5), o caminho publicado tem 621 *waypoints* e 35,8 m; o veículo percorreu 35,1 m em 95,8 s, com velocidade média de 0,37 m/s, próxima à de cruzeiro. O erro lateral, menor distância de cada amostra de odometria ao caminho planejado, teve média de 0,8 cm, raiz do erro quadrático médio de 1,3 cm, 95% das amostras abaixo de 2,9 cm e máximo de 7,1 cm, valores inferiores à tolerância de *waypoint* de 12 cm; os maiores desvios concentram-se nas mudanças de direção. Por ser calculado em relação à mesma odometria usada pelo controle, esse erro mede a qualidade do seguimento, e não a localização absoluta, que em um robô real incluiria a deriva da odometria.

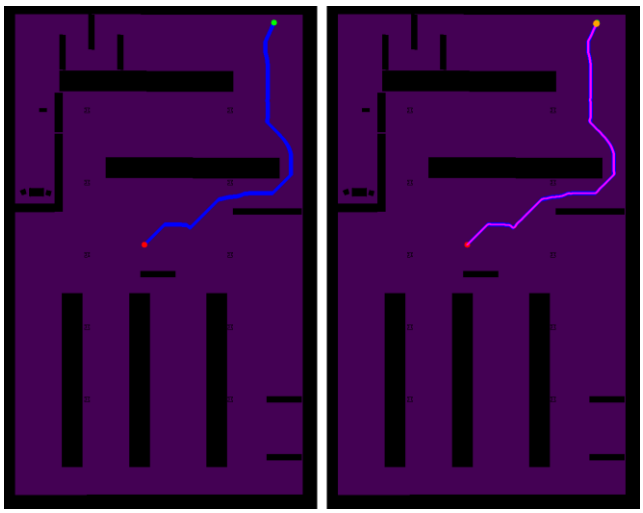


Fig. 5 – Percurso no mundo warehouse: (a) caminho planejado pelo A sobre o mapa de ocupação; (b) rastro da odometria do UGV simulado (magenta) sobreposto ao caminho.*

O fator que mais influenciou os resultados foi a correspondência entre referenciais: com a origem incorreta, o robô primeiro se desloca até o primeiro *waypoint*, e o percurso aparece deslocado em relação ao mapa. O conversor de mundos, ao fornecer a origem exata, e o reposicionamento opcional do veículo no primeiro *waypoint* resolveram o problema.

5.4 Limitações

A validação com um veículo físico, prevista inicialmente, não foi realizada por restrições de disponibilidade de plataforma, de espaço de ensaio com captura aérea associada e de prazo. O planejamento é estático, sem replanejamento diante de obstáculos que surjam durante a execução, e os resultados do RRT correspondem a uma única semente do gerador pseudoaleatório. Na configuração em WSL2, a interface gráfica do Gazebo não funcionou de forma confiável, e os percursos foram registrados a partir da

odometria, com o simulador em modo *headless*, o que não afeta a simulação em si.

6 Conclusões

Este trabalho levou o planejamento de trajetória sobre grafos, demonstrado em [2] para imagens já segmentadas, até a execução do caminho por um UGV simulado a partir de imagens aéreas reais. A arquitetura modular permitiu incorporar a segmentação semântica com modelos U-Net, comparar cinco planejadores sobre os mesmos mapas e integrar o sistema ao ROS 2 e ao Gazebo, e o conversor de mundos SDF garantiu a correspondência métrica entre o mapa de planejamento e o ambiente simulado.

Os resultados confirmam o A* e o Dijkstra como as escolhas de menor custo, com o A* explorando menos vértices, mas sem ganho de tempo nos grafos densos avaliados; a BFS produz o caminho mais curto e mais arriscado, e a DFS é inadequada. O RRT planeja de 9 a 43 vezes mais rápido, ao custo de caminhos irregulares, menos adequados a um veículo de tração diferencial. O veículo simulado concluiu todos os percursos, com erro lateral médio inferior a 1 cm no mundo simulado.

Como as interfaces entre o planejamento e o veículo são as mesmas de um robô físico, a validação em ambiente real é a continuidade imediata. Outras perspectivas são o replanejamento em ambientes dinâmicos, para o qual variantes do RRT são candidatas naturais [13]; um processo decisório que escolha ou alterne o algoritmo a partir das características da imagem de entrada [14]; e o uso de nuvens de pontos, que acrescentam elevação e declividade à avaliação de transitabilidade [15].

Agradecimentos

Os autores agradecem ao Maj Fábio Luiz Junior, cujos modelos de segmentação e trabalho de referência fundamentam este projeto, e aos autores do PFC 2025, João Vitor Camelo de Almeida e Thiago Calheiros de Souza Barbosa, cujo trabalho serviu de base para o presente projeto.

Referências

[1] F. Luiz Junior, “Cooperação entre plataformas robóticas terrestres e aéreas para aquisição, rastreamento e perseguição de alvos, utilizando visão computacional e inteligência artificial,” Tese de doutorado, Instituto Militar de Engenharia, Rio de Janeiro, 2024.

[2] J. V. C. Almeida e T. C. S. Barbosa, “Planejamento de trajetória para robô terrestre, utilizando otimização em grafos,” Projeto de Final de Curso (Graduação em Engenharia da Computação), Instituto Militar de Engenharia, Rio de Janeiro, 2025.

[3] F. Luiz Junior, “Semantic segmentation models,” repositório GitHub, 2024. [Online]. Disponível: https://github.com/fabioluiz1987/semantic_segmentation_models

[4] N. Koenig e A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), Sendai, Japão, 2004, vol. 3, pp. 2149–2154.

[5] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler e A. Y. Ng, “ROS: an open-source Robot Operating System,” in ICRA Workshop on Open Source Software, Kobe, Japão, 2009.

[6] O. Ronneberger, P. Fischer e T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in Medical Image Computing and Computer-Assisted Intervention (MICCAI), LNCS, vol. 9351. Cham: Springer, 2015, pp. 234–241.

[7] Institute of Computer Graphics and Vision, TU Graz, “Semantic Drone Dataset,” 2019. [Online]. Disponível: http://dronedataset.icg.tugraz.at/

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, Introduction to Algorithms, 3. ed. Cambridge, MA: MIT Press, 2009.

[9] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[10] P. E. Hart, N. J. Nilsson e B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[11] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Iowa State University, Ames, IA, Tech. Rep. TR 98-11, 1998.

[12] F. Luiz Junior, “RRT sobre mapa de navegação,” repositório GitHub, 2025. [Online]. Disponível: https://github.com/fabioluiz1987/RRT_sobre_mapa_de_navegacao

[13] T. Xu, “Recent advances in rapidly-exploring random tree: A review,” Heliyon, 2024.

[14] J. R. Rice, “The algorithm selection problem,” Advances in Computers, vol. 15, pp. 65–118, 1976.

[15] R. B. Rusu e S. Cousins, “3D is here: Point Cloud Library (PCL),” in Proc. IEEE Int. Conf. Robotics and Automation (ICRA), Shanghai, China, 2011, pp. 1–4.